

COMP 264: Introduction to Computer Systems (Section 001) Spring 2019
 R. I. Greenberg, Comp. Sci. Dept., Loyola U., 1052 W. Loyola Ave., Chicago, IL

Sample Exam I (100 pts)

NO BOOKS, NOTES, OR CALCULATORS ARE ALLOWED

Name _____

Assume correct implementations of the functions declared (but not defined) below in accordance with the comments. Note that the results are supposed to be independent of whether the machine is big endian or little endian, so we are assuming that any function that takes an argument composed of multiple bytes will assemble the bytes in the correct order. The functions printing in binary, octal, and hex just tell what the resulting bit pattern is in the appropriate format. On the other hand, `print_decimal` prints a short integer value using the typical format of an optional + or minus sign and then the magnitude, assuming that the argument is in two's complement notation. Also make the following assumptions:

- A short int occupies 16 bits.
- A float is 32 bits using the IEEE single precision standard (sign bit, then 8 bit exponent with bias 127, and stored significand that follows implicit "1.").
- A char is 8 bits. (Recall that a char can be used like a very small integer.)
- The argument to the function `btoi` is in nonnegative binary notation, and there is no need here to worry about the result of the function overflowing the variable it is assigned to; rather one can think of it as just copying a specified bit pattern into a variable.
- Right shifts on *signed* data are arithmetic; all other shifts are logical.

In the spaces provided, show what is printed by each of the indicated lines in the following program.

```
main ()
{
  long btoi (const char*); /* Convert binary no. in string form to decimal value */

  void print_float (double); /* Print a float in decimal */

  void print_decimal (short int); /* Print a short integer in decimal */

  void print_octal (short int); /* Print a short integer as six octal digits */

  void print_hex (short int); /* Print a short int as four hexadecimal digits */

  void print_binary_char (char); /* Print a char in binary */

  void print_binary_short_int (short int); /* Print a short int in binary */

  void print_binary_float (float); /* Print IEEE binary rep. of a float */

  short int i,j,k,l,m;
```

```
signed char x,y;
```

```
float f;
```

```
i=29873;
```

```
print_hex(i);
```

EX1-1 (8 points)

```
j=17832;
```

```
print_octal(j);
```

EX1-2 (8 points)

```
k=btoi("0000000010110011");
```

```
print_decimal(k);
```

EX1-3 (8 points)

```
l=btoi("1100100101100010");
```

```
print_octal(l);
```

EX1-4 (8 points)

```
m=0xd5a9;
```

```
print_binary_short_int(m);
```

EX1-5 (8 points)

```
x=btoi("11001000");
```

```
y=btoi("01110101");
```

```
print_binary_char(x&y);
```

EX1-6 (8 points)

```
print_binary_char(x|y);
```

EX1-7 (8 points)

```
print_binary_char(x^y);
```

EX1-8 (8 points)

```
print_binary_char(~x);
```

EX1-9 (4 points)

```
print_binary_char(y<<2);
```

EX1-10 (8 points)

```
print_binary_char(x>>3);
```

EX1-11 (8 points)

```
print_binary_char(x&&y);
```

EX1-12 (4 points)

```
f=-9.75;
```

```
print_binary_float(f);
```

EX1-13 (12 points)

```
exit(0);
```

```
}
```